

Corrigé détaillé — Sujet d'entraînement C

Decorator et Command/MVC (avec annulation)

Exercice 1 — Decorator

Question 1 — `IterateurTrace<E>`

```
import java.util.Iterator;

public class IterateurTrace<E> implements Iterator<E> {

    private Iterator<E> it;

    public IterateurTrace(Iterator<E> it) {
        this.it = it;
    }

    public boolean hasNext() {
        return it.hasNext();
    }

    public E next() {
        E element = it.next();
        System.out.println("-> element retourne : " + element);
        return element;
    }

    public void remove() {
        it.remove();
    }
}
```

C'est la structure minimale d'un Decorator : un seul champ référençant l'objet décoré (`it`), et chaque méthode soit délègue à l'identique (`hasNext`, `remove`), soit délègue **en ajoutant un comportement supplémentaire** autour de l'appel délègué (`next`). Ici on appelle d'abord `it.next()` pour récupérer l'élément réel, on l'affiche, puis on le renvoie — c'est bien le comportement délègué qui produit la valeur, le décorateur ne fait qu'observer ce qui transite.

Question 2 — `CollectionTracee<E>`

```

import java.util.Collection;
import java.util.Iterator;

public class CollectionTracee<E> implements Collection<E> {

    private Collection<E> c;

    public CollectionTracee(Collection<E> c) {
        this.c = c;
    }

    public int size() {
        return c.size();
    }

    public boolean isEmpty() {
        return c.isEmpty();
    }

    public boolean add(E e) {
        return c.add(e);
    }

    public Iterator<E> iterator() {
        return new IterateurTrace<E>(c.iterator());
    }

    public boolean contains(Object o) {
        throw new UnsupportedOperationException();
    }

    public Object[] toArray() {
        throw new UnsupportedOperationException();
    }

    public <T> T[] toArray(T[] a) {
        throw new UnsupportedOperationException();
    }

    public boolean remove(Object o) {
        throw new UnsupportedOperationException();
    }

    public boolean containsAll(Collection<?> c2) {
        throw new UnsupportedOperationException();
    }

    public boolean addAll(Collection<? extends E> c2) {
        throw new UnsupportedOperationException();
    }

    public boolean removeAll(Collection<?> c2) {
        throw new UnsupportedOperationException();
    }

    public boolean retainAll(Collection<?> c2) {
        throw new UnsupportedOperationException();
    }

    public void clear() {
        throw new UnsupportedOperationException();
    }
}

```

Le point clé de cette question est `iterator()` : ce n'est pas la collection elle-même qui logue quoi que ce soit, c'est **l'itérateur qu'elle renvoie** qui est décoré. La collection se contente de fabriquer un `IterateurTrace<E>` autour de son propre itérateur (`c.iterator()`), exactement comme `UnmodifiableCollection` fabriquait un `UnmodifiableIterator`. C'est le même schéma de composition que celui vu en cours, simplement avec un autre comportement injecté.

Les méthodes énumérées dans l'énoncé (`size`, `isEmpty`, `add`, `iterator`) sont déléguées sans aucune restriction — contrairement à l'exercice du vrai partiel, ici on n'interdit rien, on se contente de ne pas avoir implémenté les autres méthodes (ce qui est différent d'un refus *volontaire* de les supporter, mais la consigne demande explicitement ce choix pour simplifier l'exercice).

Question 3 — Différence avec `UnmodifiableCollection`

Dans les deux cas on retrouve exactement la même architecture de Decorator (une classe qui implémente la même interface que l'objet qu'elle enveloppe, et qui délègue à cet objet). La différence est dans **la nature de ce qui est ajouté** par le décorateur :

- `UnmodifiableCollection` **restreint** ce qu'il est possible de faire : certaines méthodes, normalement fonctionnelles, sont rendues volontairement inopérantes (`UnsupportedOperationException`).
- `CollectionTracee` **enrichit** le comportement sans rien interdire : toutes les opérations supportées continuent de fonctionner normalement, mais une action supplémentaire (l'affichage d'une trace) est insérée autour de l'appel délégué.

Les deux sont des decorators légitimes — le pattern ne préjuge pas de la nature du comportement ajouté, seulement de la façon dont il est structuré (même interface, délégation vers un objet enveloppé, redéfinition ciblée de certaines méthodes).

Exercice 2 — Command et MVC avec annulation

Question 1 — Interface `Commande`

```
public interface Commande {  
    void executer();  
    void annuler();  
}
```

Par rapport à un Command pattern « simple » (une seule méthode `executer()`), on ajoute ici une seconde méthode abstraite `annuler()`, qui doit défaire exactement ce que `executer()` a fait. C'est une extension très courante du pattern Command en pratique (systèmes d'undo/redo dans les éditeurs de texte, les jeux, etc.).

Question 2 — `CommandeRamasserObjet`

```
public class CommandeRamasserObjet implements Commande {  
  
    private Salle salle;  
    private Inventaire inventaire;  
    private ObjetZork objet;  
    private Vue vue;  
  
    public CommandeRamasserObjet(Salle salle, Inventaire inventaire, ObjetZork objet, Vue vue) {  
        this.salle = salle;  
        this.inventaire = inventaire;  
        this.objet = objet;  
        this.vue = vue;  
    }  
  
    public void executer() {  
        salle.retirerObjet(objet);  
        inventaire.ajouter(objet);  
        vue.afficherMessage("Vous avez ramasse " + objet.getNom() + ".");  
    }  
  
    public void annuler() {  
        inventaire.retirer(objet);  
        salle.ajouterObjet(objet);  
        vue.afficherMessage("Vous reposez " + objet.getNom() + ".");  
    }  
}
```

`annuler()` reprend exactement les mêmes données que `executer()` (la commande les a retenues dès sa construction), mais effectue les opérations dans l'ordre et le sens inverses : ce qui a été retiré de la salle y est remis, ce qui a été ajouté à l'inventaire en est retiré. C'est cette symétrie qui garantit que l'état du jeu après un `executer()` suivi d'un `annuler()` est identique à l'état initial.

Question 3 — Contrôleur avec pile d'historique

```
import java.util.ArrayDeque;
import java.util.Deque;

public class Controleur {

    private Deque<Commande> historique;

    public Controleur() {
        this.historique = new ArrayDeque<Commande>();
    }

    public void executer(Commande commande) {
        commande.executer();
        historique.push(commande);
    }

    public void annulerDerniere() {
        if (!historique.isEmpty()) {
            Commande derniere = historique.pop();
            derniere.annuler();
        }
    }
}
```

On utilise une `Deque<Commande>` comme pile (`push` / `pop`), ce qui est la structure naturelle pour un historique de type « annuler la dernière action » : la dernière commande exécutée doit être la première à pouvoir être annulée (ordre LIFO — *Last In, First Out*). `push` ajoute en tête, `pop` retire et renvoie l'élément en tête, ce qui correspond exactement à « la dernière commande exécutée ».

Question 4 — Pourquoi `annuler()` doit être dans l'interface

`Controleur` manipule les commandes uniquement à travers le type `Commande` — c'est tout l'intérêt du polymorphisme dans ce pattern : `annulerDerniere()` doit pouvoir appeler `annuler()` sur **n'importe quelle** commande dépilée, sans avoir besoin de savoir de quelle classe concrète il s'agit. Si `annuler()` n'était définie que sur certaines classes concrètes, `Controleur` devrait tester le type réel de chaque commande (avec `instanceof` et un transtypage), ce qui romprait complètement le principe du pattern Command — et plantrait ou se comporterait de façon incohérente dès qu'une commande sans `annuler()` se trouverait dans l'historique.

Pièges fréquents

1. **Décorer la collection au lieu de décorer l'itérateur qu'elle renvoie** : le piège classique de `CollectionTracee` est de vouloir logger directement dans les méthodes de la collection, alors que la trace doit apparaître à chaque élément retourné par l'itération — donc le comportement à ajouter se trouve naturellement dans `IterateurTrace.next()`, pas dans `CollectionTracee`.
2. **Oublier de déléguer `hasNext()` / `remove()`** dans le décorateur d'itérateur, ou pire, tenter de réimplémenter leur logique au lieu de simplement renvoyer/appeler ce que renvoie/fait l'itérateur décoré.
3. **`executer()` et `annuler()` non symétriques** : un oubli fréquent est de ne pas remettre l'objet exactement comme avant (par exemple oublier de retirer l'objet de l'inventaire dans `annuler()`), ce qui casse la garantie d'un état identique après un cycle exécuter/annuler.
4. **Utiliser une simple `List` avec `add/remove(size-1)` au lieu d'une vraie pile** : ça fonctionne, mais `Deque` avec

`push / pop` exprime beaucoup plus clairement l'intention (pile LIFO) et évite les erreurs d'indices.